Напиши код и интегрируй его

Ниже — **интегрированный код**: асинхронные агенты, обработка ошибок, координатор и механизм против усреднения. Ключевая идея: координатор не вычисляет «средний вердикт», а сохраняет **различие позиций** и эскалирует конфликт.

1. `src/les/agents/base_agent.py`

```
import asyncio
import logging
from abc import ABC, abstractmethod
from typing import Dict, Any

logger = logging.getLogger(__name__)


class BaseAgent(ABC):
    def __init__(self, name: str, threshold: float = 0.3):
        self.name = name
        self.threshold = threshold

    @abstractmethod
    async def evaluate(self, state: Dict[str, float]) -> Dict[str, Any]:
        """Оценивает состояние и возвращает диагноз."""

    @abstractmethod
    async def act(self, state: Dict[str, float]) -> Dict[str, Any]:
        """Принимает решение на основе оценки."""

    async def safe_act(self, state: Dict[str, float]) -> Dict[str, Any]:
        """Изолирует сбой одного агента от всего роя."""
        try:
            return await self.act(state)
        except asyncio.CancelledError:
            raise
        except Exception as exc:
            logger.exception("Агент %s завершился с ошибкой", self.name)
            return {
                "agent": self.name,
                "action": "error",
                "reason": f"{type(exc).__name__}: {exc}",
                "recoverable": True,
            }
```

2. `src/les/agents/diversity_agent.py`

Это главный агент против усреднения: он не требует «средней оценки», а проверяет, не стёрта ли индивидуальная различимость результатов.

```
from typing import Dict, Any
from .base_agent import BaseAgent


class DiversityAgent(BaseAgent):
    """
    Контур D: защищает различие.
    Усреднение допустимо только как статистика,
    но не как замена индивидуальной оценки.
    """

    def __init__(self, name: str = "diversity", threshold: float = 0.3):
        super().__init__(name, threshold)

    async def evaluate(self, state: Dict[str, float]) -> Dict[str, Any]:
        d = state.get("D", 0.0)
        return {
            "agent": self.name,
            "value": d,
            "risk": d < self.threshold,
        }

    async def act(self, state: Dict[str, float]) -> Dict[str, Any]:
        diagnosis = await self.evaluate(state)

        if diagnosis["risk"]:
            return {
                "agent": self.name,
                "action": "block",
                "reason": "Разнообразие ниже порога: индивидуальные различия стёрты"
                "value": diagnosis["value"],
            }

        return {
            "agent": self.name,
            "action": "continue",
            "reason": "Различие сохранено",
            "value": diagnosis["value"],
        }
```

3. `src/les/agents/__init__.py`

```
from .base_agent import BaseAgent
from .survival_agent import SurvivalAgent
from .others_agent import OthersAgent
from .meaning_agent import MeaningAgent
from .procedure_agent import ProcedureAgent
from .adaptation_agent import AdaptationAgent
from .diversity_agent import DiversityAgent
```

```
__all__ = [
    "BaseAgent",
    "SurvivalAgent",
    "OthersAgent",
    "MeaningAgent",
    "ProcedureAgent",
    "AdaptationAgent",
    "DiversityAgent",
]
```

4. `src/les/swarm/swarm_manager.py`

```
import asyncio
import logging
from typing import Dict, List, Any

from les.agents import BaseAgent

logger = logging.getLogger(__name__)


class SwarmManager:
    """
    Координатор роя.

    Принцип:
    - не усредняет решения агентов;
    - сохраняет все позиции;
    - выявляет конфликты;
    - эскалирует или останавливает систему.
    """

    def __init__(self, agents: list[BaseAgent]):
        self.agents = agents
        self.log: List[dict] = []

    async def run(self, state: Dict[str, float]) -> Dict[str, Any]:
        tasks = [agent.safe_act(state) for agent in self.agents]

        # Ошибки не должны обрывать работу остальных агентов
        raw_results = await asyncio.gather(*tasks, return_exceptions=True)

        results: List[dict] = []
        errors: List[dict] = []

        for agent, result in zip(self.agents, raw_results):
            if isinstance(result, BaseException):
                errors.append({
                    "agent": agent.name,
                    "error": f"{type(result).__name__}: {result}",
                })
                logger.error("Ошибка агента %s: %s", agent.name, result)
            else:
                results.append(result)
```

```
                self.log.append(result)

        conflicts = [
            result for result in results
            if result.get("action") in {"escalate", "block"}
        ]

        decision = self.decide(results, conflicts, errors)

        return {
            "state": state,
            "results": results,
            "conflicts": conflicts,
            "errors": errors,
            "decision": decision,
        }

    def decide(
        self,
        results: List[dict],
        conflicts: List[dict],
        errors: List[dict],
    ) -> str:
        if any(r.get("action") == "block" for r in results):
            return "STOP"

        if errors:
            return "ESCALATE"

        if conflicts:
            return "ESCALATE"

        return "CONTINUE"

    def get_log(self) -> List[dict]:
        return self.log
```

5. `src/les/swarm/communication.py`

```
from typing import Dict, Any, List


class CommunicationBus:
    """Шина сообщений: сохраняет позицию каждого агента без усреднения."""

    def __init__(self):
        self.messages: List[dict] = []

    def send(self, sender: str, payload: Dict[str, Any]) -> None:
        self.messages.append({
            "sender": sender,
            "payload": payload,
        })
```

```
    def receive(self) -> List[dict]:
        return self.messages.copy()
```

6. `src/les/swarm/__init__.py`

```
from .swarm_manager import SwarmManager
from .communication import CommunicationBus

__all__ = ["SwarmManager", "CommunicationBus"]
```

7. `main.py`

```
import asyncio
import logging

from les.agents import (
    SurvivalAgent,
    OthersAgent,
    MeaningAgent,
    ProcedureAgent,
    AdaptationAgent,
    DiversityAgent,
)
from les.swarm import SwarmManager

logging.basicConfig(level=logging.INFO)


async def main():
    agents = [
        SurvivalAgent("survival"),
        OthersAgent("others"),
        MeaningAgent("meaning"),
        ProcedureAgent("procedure"),
        AdaptationAgent("adaptation"),
        DiversityAgent("diversity"),
    ]

    manager = SwarmManager(agents)

    # Состояние, в котором контур D подавлен:
    # всем пытаются назначить «среднюю» оценку
    state = {
        "S": 0.8,  # выживание
        "O": 0.8,  # другие
        "M": 0.8,  # смысл
        "P": 0.8,  # процедура
        "A": 0.8,  # адаптация
        "D": 0.2,  # разнообразие — критично низкое
    }

    result = await manager.run(state)
```

```
    print("Решение координатора:", result["decision"])
    print("Конфликты:", result["conflicts"])
    print("Ошибки:", result["errors"])

    for item in result["results"]:
        print(f"{item['agent']}: {item['action']} — {item.get('reason', '')}")


if __name__ == "__main__":
    asyncio.run(main())
```

Что здесь интегрировано

- **Асинхронность:** все агенты запускаются одновременно через `asyncio.gather`.
- **Изоляция ошибок:** сбой одного агента не ломает остальных; `CancelledError` пробрасывается корректно.
- **Анти-усреднение:** `DiversityAgent` блокирует решение, если различие между индивидуальными результатами исчезло.
- **Координация без среднего:** `SwarmManager` не считает среднее мнение, а различает `CONTINUE`, `ESCALATE` и `STOP`.
- **Процедурная защита:** конфликт не «решается голосованием», а фиксируется и эскалируется.

Ожидаемый результат

При `D = 0.2` система вернёт:

```
Решение координатора: STOP
Конфликты: [{'agent': 'diversity', 'action': 'block', ...}]
```

То есть рой не позволит «средней оценке» заменить индивидуальную различимость: усреднение распознаётся как системный сбой, а не как компромисс. Это соответствует принципу, что для групповой оценки нужно оценивать не только общий результат, но и индивидуальный вклад; иначе возникает несправедливость и размывание ответственности. [1] [2] [3] [4]

⁂

1. https://www.monash.edu/learning-teaching/teachhq/Assessment/assessment-tasks/Design-and-manage-group-assessments
2. https://www.cmu.edu/teaching//designteach/design/instructionalstrategies/groupprojects/assess.html
3. https://www.ascd.org/el/articles/planning-for-fair-group-work
4. https://www.concordia.ca/ctl/resources/assessment-grading/grading-group-work.html